数据库管理系统

——项目中期报告——

组长： 敖翔

组员： 程瑶

刘烨

陈梦茹

2026 年 4 月 28 日

目录

1 项目管理 **2**

1.1 任务计划与进度演进 2

1.2 成员工作职责细分 2

2 项目方案设计 **3**

2.1 系统架构深度解析 3

2.2 中期核心成果产出 4

3 后续研发重心与演进计划 **5**

1 项目管理

1.1 任务计划与进度演进

本项目在开发过程中采用了分层递进与敏捷迭代相结合的策略。整个开发周期被逻辑性地划分为三个核心阶段：系统底层架构的搭建、核心执行引擎（DDL/DML/DQL）的实现，以及索引、事务与安全性管理等扩展功能的框架预留与简化实现。

在项目启动的前两周，团队集中力量完成了基于 Go 语言（1.22+ 版本）的架构蓝图设计，确立了系统的分层标准，并成功构筑了依托于 `dbms_root` 目录的二进制块存储引擎。进入第三至第四周后，研发重心向解析与执行层转移。期间，项目组克服了复杂语法的解析难题，落地了纯手工编写的递归下降解析器，不仅实现了数据库及数据表的基础定义操作，还同步完成了索引持久化存储（预留 `.ix` 接口）、原子性事务控制（BEGIN/COMMIT/ROLLBACK）以及基于用户权限的安全性管理模块。值得一提的是，得益于前期扎实的底层设计，基础的增删改查（DML）功能及内部的标记删除（Mark-and-Sweep）策略已提前完成联调。整个推进过程中，团队坚持以每周技术研讨的形式，持续打磨词法分析器与底层执行单元之间的接口契合度，确保了数据流转的准确与高效。

1.2 成员工作职责细分

为充分发挥团队协作效能，小组成员基于系统的高度解耦架构，进行了细致且互补的技术分工。

敖翔主要统筹底层的二进制存储引擎与基础数据执行模块。他主导了物理存储结构的设计与落地，实现了基于数据块的文件读写与行记录的精准存取。为了保证系统在内存与外存交互时的高效性，他制定并实现了基础数据类型（如 INTEGER、VARCHAR、DATETIME 等）的序列化方案及严格的四字节内存对齐规则。此外，他还承担了底层 DML 执行接口的封装工作。

程瑶专注于元数据管理体系与表结构变更逻辑的开发。她从零构建了 Catalog 数据字典模块，负责 `.tdf` 表定义文件的实时解析与动态维护。在核心的 DDL 执行层面，她承担了 `ALTER` 语句内关于字段增添、删除及类型修改的核心逻辑编写。在应对表结构动态变更时，她通过精巧的映射机制，有效保障了历史遗留数据在后续读写操作中的兼容性与一致性。。

刘烨主要负责高阶 SQL 语法解析及数据完整性约束的实现。依托手写的递归下降解析算法，他细化了抽象语法树（AST）的构建粒度，重点攻克了带有复杂条件的 `WHERE` 子句及多段式 `ALTER` 语句的精准解析。在数据规范性方面，他独立开发了 UNIQUE、NOT NULL 等系统级完整性检查模块。此外，他还对底层的字段维护代码进行了深度的结构性重构，以适配后续的扩展需求。

陈梦茹主导了查询解析层的逻辑梳理与系统级调度控制。他着力于 `SELECT` 查询引擎的深层次构建，通过搭建支持 AND、OR 及比较运算的 Condition 条件语法树，实现

了复杂条件下的数据精准过滤。同时，他向上统筹了各类 DML 操纵语句的系统级调度，保障了并发访问逻辑的清晰。在扩展功能方面，他完成了系统的基础自动化测试闭环，并实现了完整的安全性管理模块，支持 `CREATE USER` 及针对表级的 `GRANT` 权限授予功能，同时完成了数据库 `BACKUP` 与 `RESTORE` 的维护逻辑开发。

贡献度：敖翔：25% 程瑶：25% 刘烨：25% 陈梦茹：25%

2 项目方案设计

2.1 系统架构深度解析

Ruanko DBMS 的设计哲学体现在其严谨的四层流水线架构中。这种模式在确保模块内部逻辑高度内聚的同时，实现了层与层之间的清晰边界与深度解耦。

系统的工作流始于**解析层（Parser）**。用户输入的原始 SQL 字符串在此处经由词法分析，并被手工编写的递归下降解析器处理，最终转化为结构化的抽象语法树（AST）。随后，**执行层（Executor）**接管这棵语法树。它扮演着核心调度者的角色，通过分析 AST 节点的语义，按需调用元数据接口或下发存储引擎指令，以驱动实际的业务逻辑。

在向下延伸的数据管理枢纽处，**元数据与存储层（Catalog/Storage）**承担了双重使命：一方面，它负责解析和维护由 `.tdf` 和 `.tb` 等文件构成的数据库元数据信息；另一方面，它构建了面向二进制文件的高效块存储抽象层。最终，所有的数据变换均忠实地落地于底层的**物理文件层**，通过 `.trd` 记录文件与配套的日志文件，确保了数据的持久化与安全性。

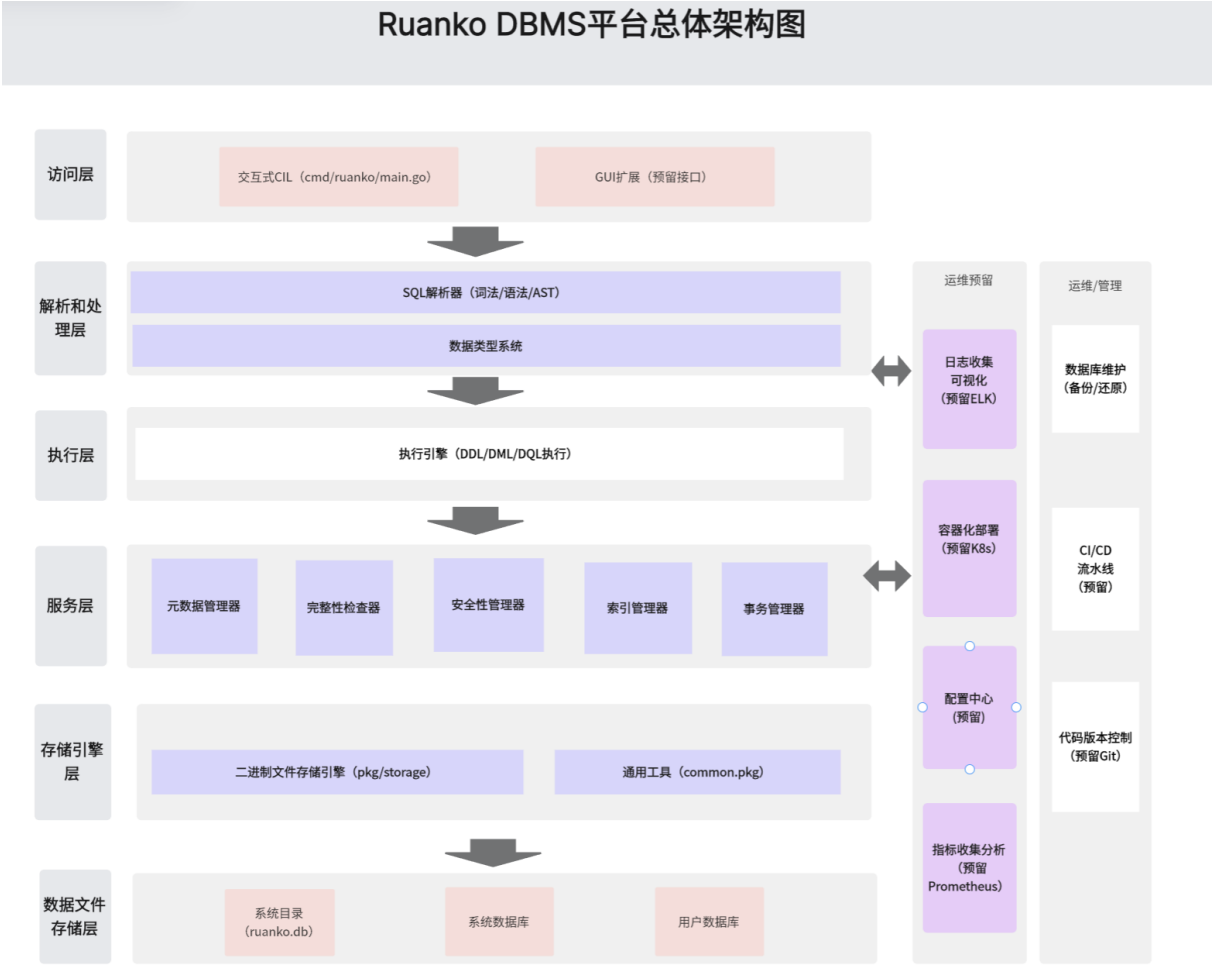

图 1: DBMS 系统架构图

2.2 中期核心成果产出

对照立项之初设定的考核标准，系统在核心功能栈的建设上已达成显著的阶段性里程碑：

首先，在定义语言（DDL）层面，系统现已全面打通了涵盖创建、删除及修改的完整解析与执行链路。数据库层面支持物理目录的隔离分配与状态切换；数据表层面不仅实现了建表与删表机制，更支持通过持久化的完整性约束（如主键和非空限制）来保障数据质量。在灵活度要求极高的字段级修改上，系统能够平滑处理 `ADD`、`MODIFY` 与 `DROP COLUMN` 指令，并在后台自动重构二进制数据的对齐方案。

其次，在数据操作层面，项目组通过高强度的协同攻关，超额交付了 DML 与 DQL 的基础版本。系统不仅能够稳定执行记录的插入、更新与标记删除，还成功引入了支持复合条件过滤（包含比较运算符与逻辑连接符）的精确查询机制。

最后，在高级特性方面，系统已成功集成内存与文件同步的索引映射框架（预留 `.ix` 接口），并落地了支持原子性语义的事务管理引擎。安全性方面，系统构建了基于用户角色的权限控制体系，确保了多用户环境下的数据安全。

3 后续研发重心与演进计划

在接下来的开发周期中，系统演进的重心将从基础框架的搭建转向高级特性的深度优化与持久化逻辑的打通。

首要任务是推进并完善索引管理体系。目前，系统内部已预留了内存索引映射框架。我们计划在此基础上，进一步打通针对 .ix 文件的持久化存储接口，以期在未来数据量膨胀时，依然能维持查询性能的相对稳定。

在提升系统运行的健壮性方面，目前系统已实现了简化版的事务控制框架，支持 BEGIN、COMMIT 和 ROLLBACK 指令的基础调度。后续我们将进一步强化事务管理器的原子性语义，通过完善日志机制（.log）来确保在异常中断时能够利用事务逻辑保护数据的完整性，避免脏写问题。

此外，得益于前期良好的代码分层设计，当前的所有核心数据库逻辑均被紧凑地封装在 pkg/ 包内，现有的 CLI 入口仅充当了一层轻量级的调用外壳。这种设计为我们后续剥离命令行、开发跨平台的图形化管理界面（GUI）预留了充足的空间。通过图形化界面的扩展，我们期望能显著降低系统的交互门槛，提供更加直观且现代化的数据库管理体验。